

Лабораторна робота №2.

Планування завдань у FreeRTOS

Мета роботи. Ознайомитися з принципами планування завдань у FreeRTOS.

Теоретичні відомості.

Завдання (tasks) – це незалежні потоки виконання, які планувальник RTOS виконує у режимі псевдопаралельності. FreeRTOS підтримує пріоритетне витісняюче планування:

- Завдання з вищим пріоритетом витісняє завдання з нижчим.
- Якщо кілька завдань мають однаковий пріоритет, вони виконуються по черзі (round-robin).

Функція для створення завдання:

```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode, // функція завдання  
    const char * const pcName, // ім'я завдання  
    configSTACK_DEPTH_TYPE usStackDepth, // розмір стеку  
    void *pvParameters,        // параметри  
    UBaseType_t uxPriority,     // пріоритет  
    TaskHandle_t *pxCreatedTask // хендл  
);
```

Хід виконання роботи.

1. Створіть проєкт FreeRTOS у середовищі розробки VS Code.
2. Скомпілюйте та завантажте наведений код у Pico.

1	#include <stdio.h>
2	#include "pico/stdlib.h"
3	#include "FreeRTOS.h"
4	#include "task.h"
5	
6	void vTask1(void *pvParameters) {
7	while (1) {
8	printf("Task 1 (Low priority)\n");

```

9     vTaskDelay(pdMS_TO_TICKS(1000));
10 }
11 }
12
13 void vTask2(void *pvParameters) {
14     while (1) {
15         printf("Task 2 (Medium priority)\n");
16         vTaskDelay(pdMS_TO_TICKS(700));
17     }
18 }
19
20 void vTask3(void *pvParameters) {
21     while (1) {
22         printf("Task 3 (High priority)\n");
23         vTaskDelay(pdMS_TO_TICKS(500));
24     }
25 }
26
27 int main() {
28     stdio_init_all();
29
30     xTaskCreate(vTask1, "Task1", 256, NULL, 1, NULL);
31     xTaskCreate(vTask2, "Task2", 256, NULL, 2, NULL);
32     xTaskCreate(vTask3, "Task3", 256, NULL, 3, NULL);
33
34     vTaskStartScheduler();
35
36     while (1) {} // сюди виконання не доходить
37 }

```

3. Спостерігайте у UART-консолі, як виконуються завдання.
4. Змініть пріоритети так, щоб:
 - всі завдання мали однаковий пріоритет,
 - пріоритети були переставлені у зворотному порядку.
5. Порівняйте результати:
 - у якому порядку виконуються завдання?
 - що відбувається, якщо завдання з високим пріоритетом не викликає `vTaskDelay()`?
6. Додайте четверте завдання з найнижчим пріоритетом, яке блимає LED кожні 2 секунди.

7. Організуйте експеримент: одне завдання з високим пріоритетом працює без `vTaskDelay()`. Поясніть, чому інші завдання не виконуються.
8. Створіть дві пари завдань з однаковими пріоритетами. Переконайтеся, що планувальник розподіляє процесорний час між ними.